Practical 9: Time and Space Complexity Analysis

Aim

To study and analyze the time and space complexity of Linear Search and Binary Search algorithms.

Theory

Time complexity measures how the running time of an algorithm grows as a function of input size (n), typically expressed using Big-O notation, which describes the worst-case upper bound on growth while ignoring constant factors and lower-order terms; common classes in increasing order of growth include O(1), O(log n), O(n), O(n log n), and O(n^2). Space complexity measures the additional (auxiliary) memory an algorithm requires beyond the input itself, which is important when working with large datasets or memory-constrained systems. Linear Search checks each element of the array sequentially until it finds a match or reaches the end, giving a worst-case time complexity of O(n) since it may need to examine every element. Binary Search works only on sorted data and repeatedly halves the search range by comparing the target with the middle element, giving a much better time complexity of O(log n), which scales far more efficiently as the dataset grows large - this is empirically demonstrated in this practical by timing both algorithms on a dataset of 100,000 elements. Both algorithms use O(1) additional space since they operate in-place using only a fixed number of variables regardless of input size, rather than allocating new data structures proportional to n.

Algorithm

1. Define linear_search() which iterates through the array checking each element against the target.
2. Define binary_search() which divides the sorted array range in half repeatedly.
3. Generate a large list of data and choose a target value.
4. Measure execution time of linear_search() using time.perf_counter().
5. Measure execution time of binary_search() similarly.
6. Print both execution times along with their theoretical time and space complexities.

Program

```
import time

def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1

def binary_search(arr, target):
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1

data = list(range(100000))
target = 99999

start = time.perf_counter()
linear_search(data, target)
t1 = time.perf_counter() - start
start = time.perf_counter()
```

```
binary_search(data, target)
t2 = time.perf_counter() - start

print(f"Linear Search Time: {t1:.6f} s  (Complexity: O(n))")
print(f"Binary Search Time: {t2:.6f} s  (Complexity: O(log n))")
print("Space Complexity: O(1) for both (in-place)")
```

Output

```
Linear Search Time: 0.002474 s  (Complexity: O(n))
Binary Search Time: 0.000005 s  (Complexity: O(log n))
Space Complexity: O(1) for both (in-place)
```

Result / Conclusion

The time and space complexity of Linear Search and Binary Search were analyzed and empirically verified, confirming Binary Search is significantly faster on large sorted datasets.

Viva Questions

Q1. What is Big-O notation and what does it represent?

Ans: Big-O notation describes the upper bound (worst-case) growth rate of an algorithm's time or space requirements as the input size n increases, ignoring constant factors and lower-order terms. For example, O(n) means the time grows linearly with n; O(n²) means it grows quadratically. It helps compare algorithms independently of hardware or implementation details. Other notations: Ω (best case), Θ (tight/average case).

Q2. Why does Binary Search have O(log n) time complexity?

Ans: Each comparison in Binary Search eliminates half the remaining elements. Starting with n elements, after 1 step n/2 remain, after 2 steps n/4, after k steps n/2^k. The algorithm terminates when 1 element remains: n/2^k = 1 ■ k = log■(n). So the maximum number of comparisons is proportional to log■(n), giving O(log n) time complexity.

Q3. What is the difference between time complexity and space complexity?

Ans: Time complexity measures how the number of operations (running time) grows with input size n. Space complexity measures how much additional memory (auxiliary space) an algorithm uses beyond the input itself, also as a function of n. An algorithm can be fast but memory-hungry (e.g., merge sort: O(n log n) time, O(n) space) or slow but memory-efficient (e.g., bubble sort: O(n²) time, O(1) space).

Q4. What does O(1) space complexity mean for both search algorithms?

Ans: O(1) space complexity means the algorithm uses a constant amount of additional memory regardless of the input size n. Both Linear Search and Binary Search only use a fixed number of scalar variables (loop counter i, or low/high/mid pointers) — no additional arrays, lists, or data structures are allocated proportional to n. The input array itself is not counted as auxiliary space.

Q5. When would you prefer Linear Search over Binary Search?

Ans: Linear Search is preferred when: (1) the data is unsorted — sorting first adds O(n log n) cost that outweighs the benefit for a single search; (2) the dataset is very small (n < ~20) — the constant overhead of Binary Search matters; (3) searching through a linked list — you cannot do random O(1) access needed for Binary Search; (4) searching on a condition other than equality (e.g., first element satisfying a predicate).